



Deployment, Inference & MLOps

Session 4 — Model Serving, Monitoring & Automated Retraining

MAY 2026 · SESSION 4



Chris Caudill

Senior AI/ML Architect

Snowflake

PRESENT

Senior AI/ML Architect

Snowflake

End-to-end ML platform strategy, customer architecture, production ML enablement.

5 YEARS

Senior ML Engineer

Amazon Web Services (AWS)

Enterprise-scale MLOps, Fuels Price Optimization, Agentic Systems.

10 YEARS

Senior Data Scientist

ExxonMobil

Investigative NLP, Market Signal Detection, Summarization Modeling.

 SERIES

Session Series: Machine Learning on Snowflake

A 4-part deep dive into production ML capabilities on Snowflake.

SESSION 1 · COMPLETE

End-to-End ML Overview

Platform overview, production-grade ML, live demo.

SESSION 2 · COMPLETE

Feature Store, Experiments & Registry

Entities, FeatureViews, point-in-time correctness, experiment tracking.

SESSION 3 · COMPLETE

Distributed Training & HPO

ML Jobs, multi-node training, hyperparameter optimization.

SESSION 4 · NOW

Deployment, Inference & MLOps

Model serving, monitoring, automated retraining.

 AGENDA

Today's Agenda

From model registry to production endpoints with full observability.



Part 1 — Deployment

- › Model Service on SPCS
- › Snowflake Gateway
- › Auto-Capture & Logging
- › Version-pinned services



Part 2 — Inference

- › REST API (real-time)
- › SQL Inference (batch)
- › Model aliases & promotion
- › Traffic splitting



Part 3 — MLOps

- › Model Monitoring & Drift
- › Auto-Capture analysis
- › Drift Alerts
- › Automated Retraining

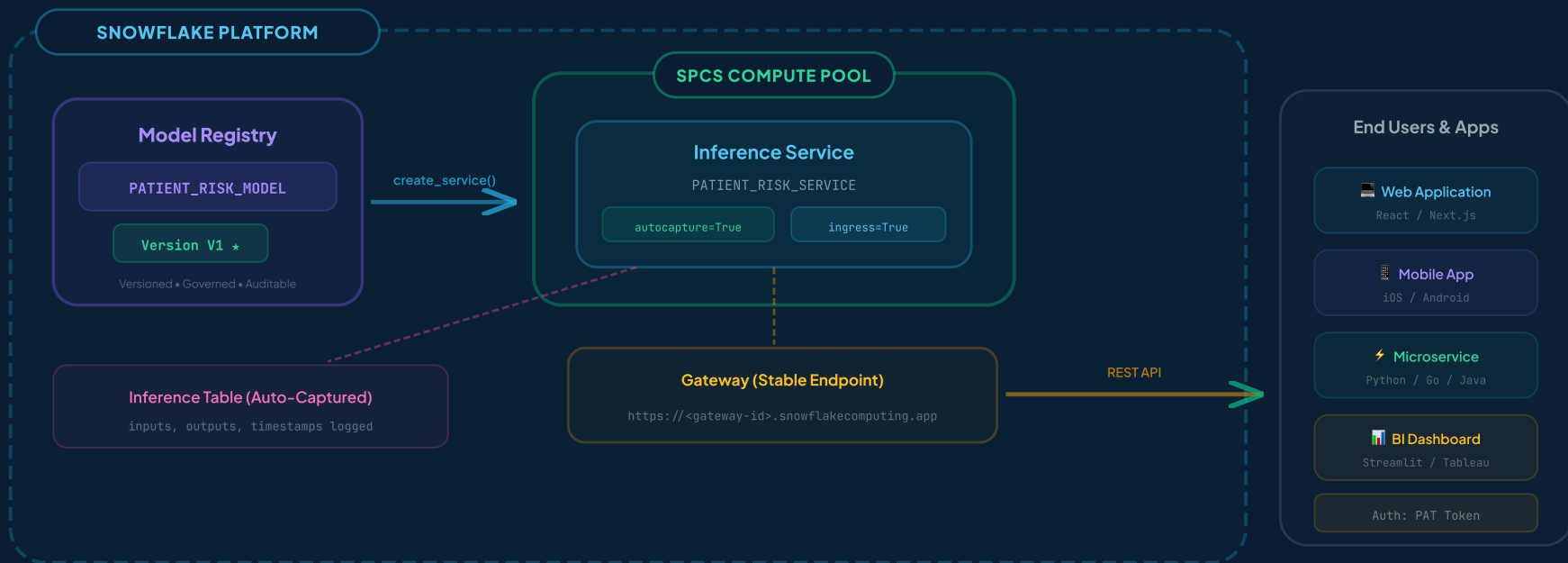
Part 1

Model Deployment

From the Model Registry to a managed REST endpoint — one API call.

Model Serving Architecture

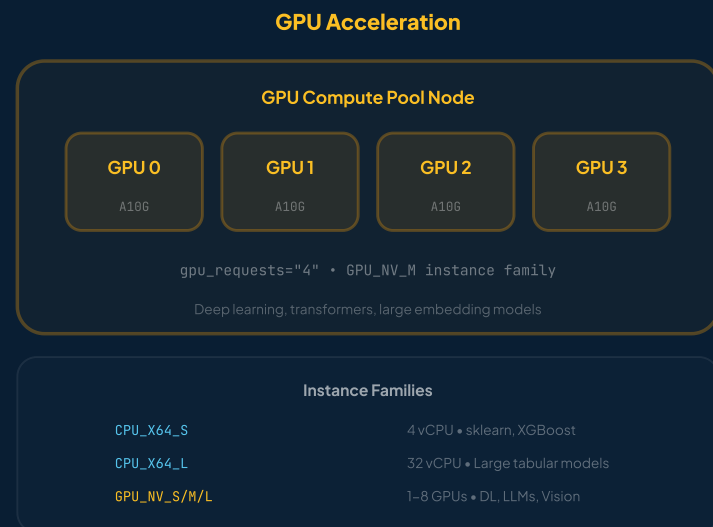
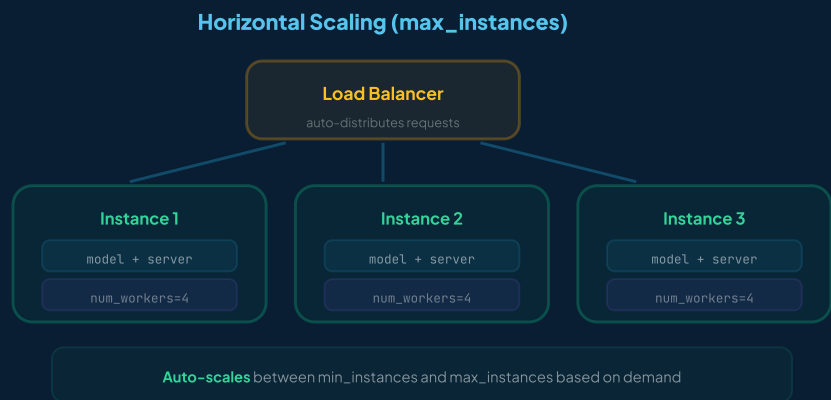
A registered model version is deployed as a managed container service within Snowflake's security perimeter, exposed to applications via a stable REST endpoint.





Service Scalability

Inference services scale horizontally with multiple instances and support GPU acceleration for compute-intensive models.





CODE

Deploying a Model **Service**

KEY PARAMETERS

service_compute_pool GPU or CPU pool

ingress_enabled Public HTTP endpoint

autocapture Log all requests

max_instances Horizontal scaling

gpu_requests GPU allocation

IMPORTANT

autocapture is **immutable** — cannot be toggled after service creation. Must recreate the service.

DEPLOY_MODEL.PY

```
from snowflake.ml.registry import Registry

reg = Registry(session, database_name="ML_DEMO_PIPELINE_DB",
               schema_name="HEALTHCARE")

# Get specific model version
mv = reg.get_model("PATIENT_RISK_MODEL").version("V1")

# Deploy as managed SPCS service
mv.create_service(
    service_name="PATIENT_RISK_SERVICE",
    service_compute_pool="ML_DEMO_COMPUTE_POOL",
    ingress_enabled=True,
    max_instances=3,
    autocapture=True,
)

# Get endpoint URL
services = mv.list_services()
endpoint = services['inference_endpoint'].iloc[0]
```

Part 2

Inference

Two paths to predictions: SQL in the warehouse, REST in the compute pool.

Two Inference Paths

SQL inference runs on your virtual warehouse. Real-time inference runs on the SPCS compute pool. Same model, different compute.





CODE

Inference Code Examples

Call the same model two ways: HTTP for real-time apps (left), SQL for batch pipelines (right).

 External apps call the REST endpoint via the Gateway

REST API (COMPUTE POOL)

```
import json, requests, pandas as pd

df = pd.DataFrame({
    "AGE": [65], "BMI": [28.5],
    "HEART_RATE": [88], ...
})

payload = {"dataframe_split":
    json.loads(df.to_json(orient="split"))}

url = "https://<gw>.snowflakecomputing.app/predict"
headers = {
    "Authorization": f'Snowflake Token="{PAT}"',
    "Content-Type": "application/json",
}

response = requests.post(
    url, json=payload, headers=headers)
result = response.json()
# {"data": [[0, {"output_feature_0": "HIGH"}]]}
```

 SQL queries reference the model directly — no service needed

SQL INFERENCE (WAREHOUSE)

```
-- Default version (simplest)
SELECT PATIENT_ID,
    PATIENT_RISK_MODEL!PREDICT(
        AGE, BMI, HEART_RATE, ...
    ) AS prediction
FROM RAW_PATIENT_DATA;

-- Specific version via alias
WITH v AS MODEL PATIENT_RISK_MODEL
    VERSION production
SELECT PATIENT_ID,
    v!PREDICT(AGE, BMI, ...) AS prediction
FROM RAW_PATIENT_DATA;

-- Batch scoring (CTAS)
CREATE TABLE PREDICTIONS AS
SELECT *, PATIENT_RISK_MODEL!PREDICT(...)
    AS prediction
FROM RAW_PATIENT_DATA;
```

Part 3

MLOps

Model promotion, monitoring, drift detection, and automated retraining.



Model Promotion with Aliases

Aliases are exclusive labels that track lifecycle stages. Only one version can hold a given alias. Use the default version for zero-change SQL inference.



PROMOTION WORKFLOW

```
ALTER MODEL ... VERSION V1
UNSET ALIAS;

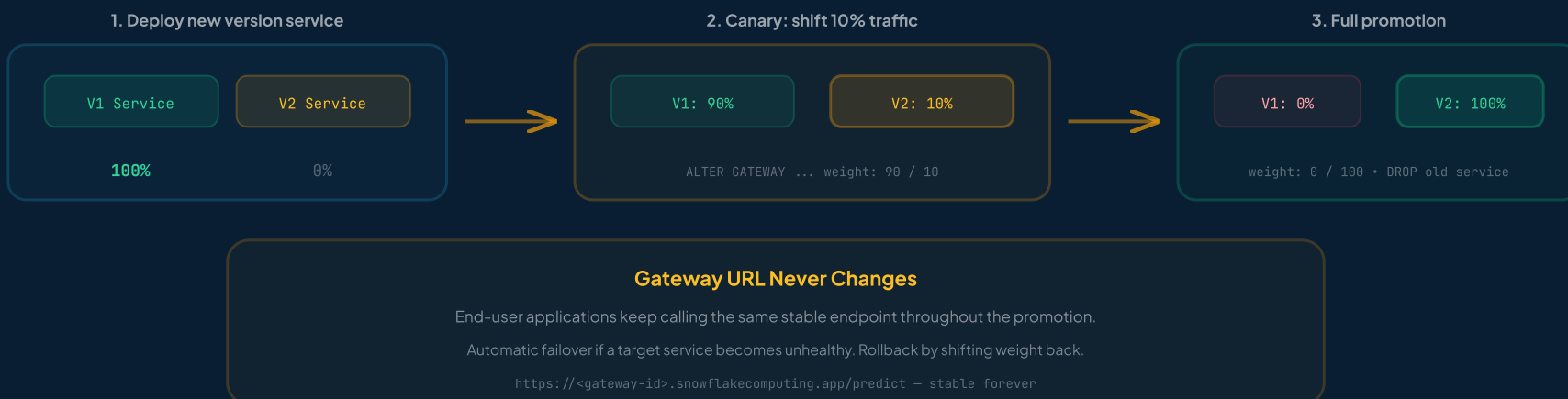
ALTER MODEL ... VERSION V2
SET ALIAS = production;

ALTER MODEL ...
SET DEFAULT_VERSION = V2;
```

SQL callers see new model instantly

Promoting Models for **Real-Time Services**

For inference services, promote new model versions by deploying a new service behind the gateway and gradually migrating traffic. Zero-downtime canary deployments.



Auto-Capture & Drift Detection

Every inference is logged. Compare production distributions against baseline to detect when the world has changed.

INFERENCE_TABLE('PATIENT_RISK_MODEL')

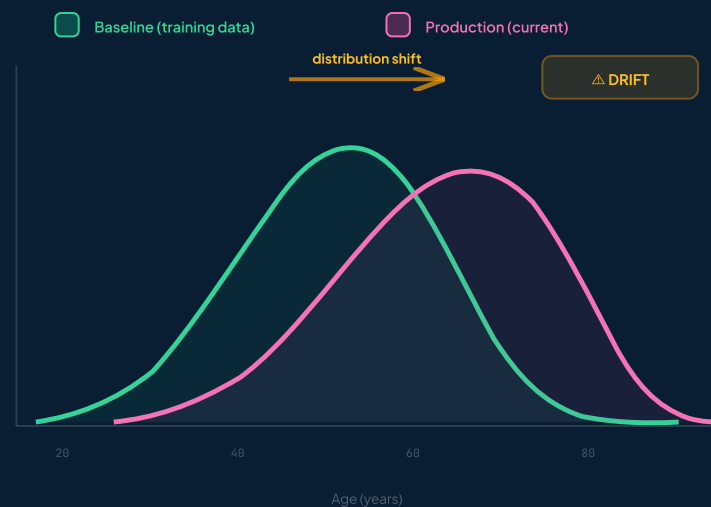
TIMESTAMP	AGE	BMI	HEART_RATE	PREDICTION
2026-05-19 08:12	72	31.2	95	HIGH
2026-05-19 08:14	45	24.8	72	LOW
2026-05-19 08:15	58	27.1	84	MEDIUM
...	...	...	...	...

How It Works

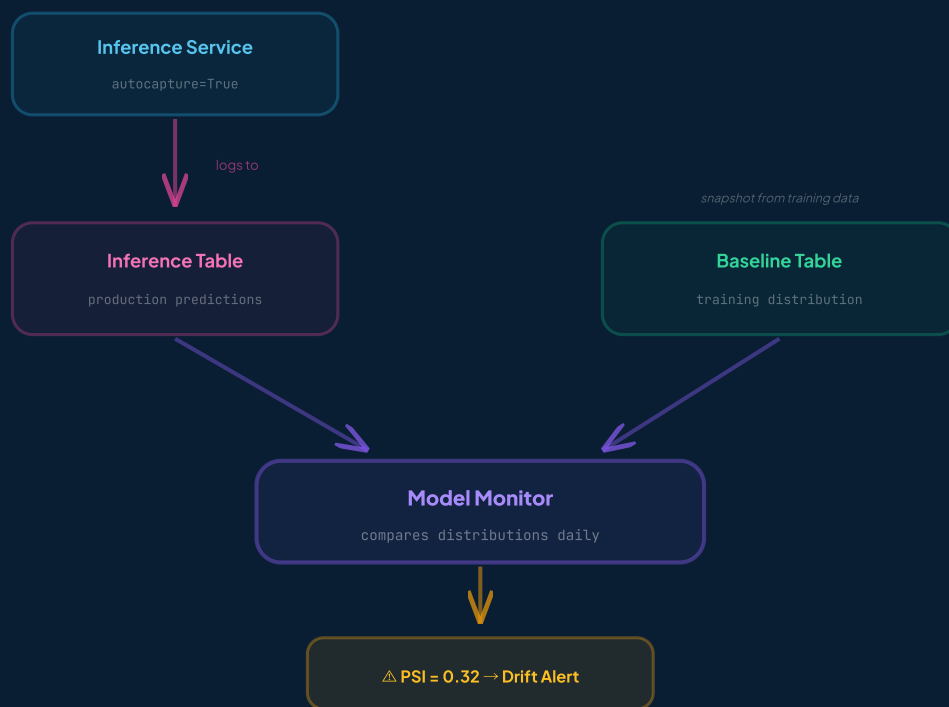
1. Service logs every request/response (Auto-Capture)
2. Model Monitor compares current data vs baseline table
3. Computes PSI (Population Stability Index) per column
4. If PSI exceeds threshold → drift detected → trigger action

PSI < 0.10: stable | 0.10–0.25: investigate | > 0.25: significant drift

Distribution Comparison: AGE



Monitoring Architecture & Drift Types



Data Drift (Covariate Shift)

Input feature distributions change. e.g., patients are older, higher BMI than training data.

Prediction Drift

Output distribution shifts. e.g., model predicts “HIGH” more often than during training.

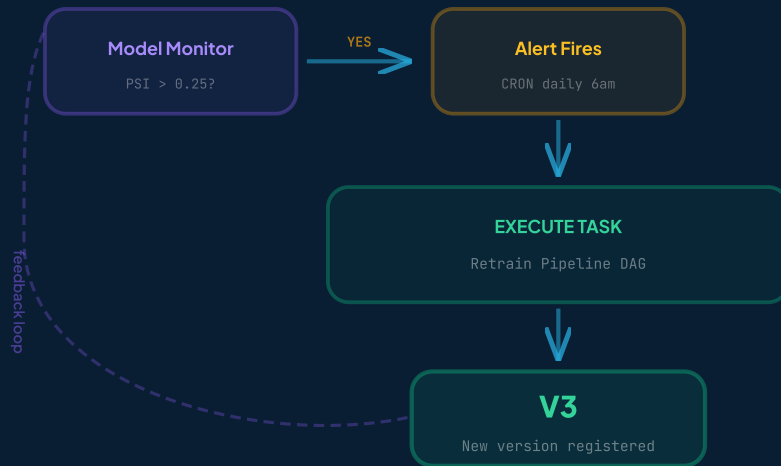
Concept Drift

The relationship between inputs and outputs changes. Same features now map to different risk levels.

AVAILABLE METRICS

PSI KL_DIVERGENCE ACCURACY F1 COUNT

Automated Retraining with Alerts



SQL – CREATE DRIFT ALERT

```

CREATE OR REPLACE ALERT
PREDICTION_DRIFT_ALERT
WAREHOUSE = ML_DEMO_WAREHOUSE
SCHEDULE = 'USING CRON 0 6 * * *
America/Los_Angeles'
IF (EXISTS (
  SELECT 1
  FROM TABLE(
    MODEL_MONITOR_DRIFT_METRIC(
      'PATIENT_RISK_MONITOR',
      'PSI', 'RISK_LEVEL', 'DAY',
      DATEADD('day', -1,
        CURRENT_TIMESTAMP())
      ::TIMESTAMP_NTZ,
      CURRENT_TIMESTAMP()
      ::TIMESTAMP_NTZ))
  WHERE "value" > 0.25
))
THEN
  EXECUTE TASK
  PIPELINE_FEATURE_ENG_TASK;

ALTER ALERT PREDICTION_DRIFT_ALERT
RESUME;
  
```

Live Demo

Deployment → Inference → Monitoring — all from Jupyter notebooks



RECAP

Session 4 Key Takeaways



Deployment

- › `create_service()` — one API call
- › Scalable: multi-instance + GPU
- › Gateway for stable endpoints
- › Auto-Capture logs everything



Inference

- › REST: compute pool (real-time)
- › SQL: warehouse (batch)
- › Aliases for lifecycle stages
- › Gateway traffic splitting



MLOps

- › Model Monitor with PSI drift
- › Data, prediction & concept drift
- › Alerts trigger retraining
- › Full feedback loop closed

 COMPLETE

Series **Complete!**

The full ML lifecycle on Snowflake — from raw data to production with automated retraining.



Session 1

End-to-End Overview



Session 2

Features & Registry



Session 3

Distributed & HPO



Session 4

Deploy & MLOps

Questions?

Thank you for attending the ML on Snowflake series!